

App concept: CleanBeat AI

CleanBeat AI is a multimodal web application that turns a photograph of a messy room into a practical cleaning session.

The user takes or uploads a room photo. The application analyzes visible clutter, identifies likely cleaning tasks, estimates a reasonable time range, organizes the work into an efficient sequence, and recommends music that matches the expected effort and duration.

A result might say:

Estimated time: 30–45 minutes

Room condition: Moderately cluttered

Suggested order:

1. Pick up clothes
2. Remove trash and dishes
3. Clear the desk
4. Organize loose objects
5. Vacuum the floor

Music: High-energy pop, approximately 125 BPM

Playlist: 40-minute Spotify cleaning playlist

The real value is not merely “AI rates how messy your room is.” That would be a gimmick. The stronger product turns an unstructured image into a structured, editable action plan.

What the application should and should not claim

The AI cannot reliably determine exactly how long cleaning will take from one photo. It cannot see behind furniture, know how quickly the user works, or determine whether objects need washing, sorting, repairing, or discarding.

Therefore, the app should provide:

- an estimated range rather than an exact number
- a confidence level
- visible assumptions
- an editable task list
- the ability for the user to adjust room size and cleaning intensity

For example:

Estimated time: 35–50 minutes

Assumes one person, standard cleaning, and no hidden clutter.

That is more credible than pretending the model knows the room will take exactly 43 minutes.

Recommended MVP

The first usable version should do only this:

1. Accept one room image.
2. Validate and preview the image.
3. Analyze it with a vision-capable AI model.
4. Return structured room-analysis data.
5. Generate an ordered cleaning checklist.
6. Estimate a cleaning-time range.
7. Recommend a music mood and Spotify playlist.
8. Let the user edit and check off tasks.
9. Deploy the app publicly.

Do not initially build:

- user accounts
- social features
- custom AI-generated music
- object detection boxes
- before-and-after scoring
- gamification
- household collaboration
- mobile apps
- subscription payments
- a separate microservice architecture

Those may make the project longer without making the initial portfolio demonstration much stronger.

Recommended technology stack

Main stack

Area	Technology	Purpose
Language	TypeScript	Shared frontend and backend type safety
Web framework	Next.js App Router	UI, server-side logic and API endpoints
UI library	React	Interactive upload and result screens

Styling	Tailwind CSS	Responsive styling without extensive custom CSS
UI components	shadcn/ui	Accessible buttons, cards, dialogs and progress elements
AI	OpenAI Responses API	Analyze room images and return structured results
Validation	Zod	Validate AI responses, forms and API input
Music	Spotify Web API	Search for matching public playlists
Deployment	Vercel	Host the Next.js application
Testing	Vitest + React Testing Library	Unit and component testing
Browser testing	Playwright	End-to-end upload and analysis testing
Monitoring	Vercel Logs initially	Diagnose production errors
Source control	GitHub	Repository, issues, documentation and CI

Next.js App Router supports pages, server and client components, server-side route handlers and deployment as one project, which is appropriate for a compact portfolio application. ([Next.js](#))

OpenAI's server-side JavaScript SDK accepts image inputs, allowing the application to submit a room image for visual analysis. ([OpenAI Platform](#))

Spotify's Web API can search its catalog for playlists, but requests require authorization. Creating or modifying a user's own playlists requires additional user permission, so that functionality should not be part of the first release. ([Spotify for Developers](#))

Architecture

For the first version, use one Next.js application rather than separate frontend and backend repositories.

Browser

```
|
| Upload image and preferences
v
```

Next.js application

```
|
|-- Validate image
|-- Call room-analysis service
|-- Validate structured AI output
```

|-- Calculate time estimate

|-- Search Spotify playlists

v

Results page

Suggested repository structure:

cleanbeat-ai/

```
├── app/
│   ├── page.tsx
│   ├── analyze/
│   │   └── page.tsx
│   ├── results/
│   │   └── page.tsx
│   └── api/
│       ├── analyze/
│       │   └── route.ts
│       └── music/
│           └── route.ts
├── components/
│   ├── image-uploader.tsx
│   ├── image-preview.tsx
│   ├── room-preferences-form.tsx
│   ├── analysis-progress.tsx
│   ├── cleaning-plan.tsx
│   ├── task-item.tsx
│   ├── time-estimate.tsx
│   └── playlist-card.tsx
├── lib/
│   ├── ai/
│   │   ├── analyze-room.ts
│   │   ├── analysis-schema.ts
│   │   └── prompts.ts
│   ├── cleaning/
│   │   ├── estimate-time.ts
│   │   └── order-tasks.ts
│   ├── spotify/
│   │   ├── spotify-client.ts
│   │   └── search-playlists.ts
│   ├── image/
│   │   └── validate-image.ts
│   └── env.ts
├── tests/
├── public/
├── .env.example
├── README.md
└── package.json
```

The important architectural decision is to keep AI communication, time calculations and Spotify communication out of the React components.

Step-by-step implementation plan

Phase 1: Define the product before coding

Step 1: Write the MVP requirements

He should create a short `docs/product-requirements.md` file.

It should define:

User story

As a user, I want to upload a photo of a messy room so that I can receive an organized cleaning plan, a realistic time estimate and music suitable for the cleaning session.

Required inputs

- room photograph
- room type:
 - bedroom
 - kitchen
 - living room
 - office
 - bathroom
 - other
- cleaning intensity:
 - quick reset
 - standard cleaning
 - deep cleaning
- optional room size:
 - small
 - medium
 - large
- optional preferred music genres

Required outputs

- room-condition summary
- clutter level
- identified tasks
- task order
- estimated total time range

- assumptions
- confidence level
- music characteristics
- Spotify playlist suggestions

Technology used

No implementation technology yet beyond Markdown and GitHub.

Completion criteria

The requirements should be no longer than roughly two pages. A giant specification would be wasteful at this stage.

Phase 2: Prove that the AI analysis works

Step 2: Define the room-analysis data contract

Before calling an AI model, define exactly what the application expects back.

Use TypeScript and Zod.

```
import { z } from "zod";

export const roomAnalysisSchema = z.object({
  roomType: z.enum([
    "bedroom",
    "kitchen",
    "living_room",
    "office",
    "bathroom",
    "other",
  ]),
  clutterLevel: z.number().int().min(1).max(10),
  summary: z.string().min(1).max(500),
  visibleConditions: z.array(z.string()).max(15),
  tasks: z.array(
    z.object({
      id: z.string(),
      title: z.string(),
      description: z.string(),
      category: z.enum([
        "trash",
        "laundry",
        "dishes",
        "organizing",
        "surface_cleaning",
      ]),
    })
  ),
});
```

```

    "floor_cleaning",
    "other",
  ]),
  estimatedMinutes: z.number().int().min(1).max(120),
  priority: z.enum(["low", "medium", "high"]),
  evidence: z.string(),
})
),
assumptions: z.array(z.string()),
uncertaintyNotes: z.array(z.string()),
confidence: z.number().min(0).max(1),
recommendedMusic: z.object({
  energy: z.enum(["low", "medium", "high"]),
  mood: z.string(),
  genres: z.array(z.string()).max(5),
  targetBpmMin: z.number().int().min(50).max(220),
  targetBpmMax: z.number().int().min(50).max(220),
}),
});

export type RoomAnalysis = z.infer<typeof roomAnalysisSchema>;

```

Why this comes first

Without a schema, the AI may return differently formatted prose on every request. That causes brittle parsing and unpredictable UI behavior.

Technology used

- TypeScript
- Zod
- Vitest

Testing

Write tests proving that:

- valid analysis passes
- missing tasks fail
- clutter levels outside 1–10 fail
- invalid confidence values fail
- nonsensical BPM values fail

Step 3: Build a local AI-analysis script

Do not start with the polished website.

Create a command-line script such as:

scripts/analyze-room.ts

The script should:

1. Read a local image.
2. Convert it into an acceptable image input.
3. Send it to the OpenAI API.
4. Request structured output.
5. Validate the response using the Zod schema.
6. Print formatted JSON.
7. Save the result to a local file for inspection.

Prompt responsibilities

The prompt should tell the model to:

- analyze only visible conditions
- avoid identifying people
- avoid inferring sensitive personal information
- distinguish visible evidence from assumptions
- recommend tasks, not judgments
- avoid insulting language such as “disgusting”
- estimate each task independently
- return a range rather than false precision
- flag uncertainty

Example prompt direction

Analyze this room as a cleaning-planning assistant.

Use only visible evidence. Do not infer the user's personality, income, health, living conditions, or personal habits.

Identify visible cleaning and organizing tasks. For every task:

- explain the visible evidence
- estimate active minutes
- assign a priority
- assign a category

Return uncertainty notes for anything that cannot be determined from the image.

Evaluation set

He should test at least:

- clean bedroom

- slightly cluttered bedroom
- heavily cluttered bedroom
- kitchen with dishes
- office with papers
- low-light image
- blurry image
- room containing a person
- image that is not a room
- screenshot or meme instead of a photograph

Technology used

- Node.js
- TypeScript
- OpenAI JavaScript SDK
- Zod
- dotenv or Next.js environment handling
- local image fixtures

Completion criteria

The AI should produce valid structured output for most normal room photos. When it cannot, the application should return a useful error instead of broken JSON.

Phase 3: Build deterministic cleaning-time logic

Step 4: Separate time calculation from AI judgment

The AI may suggest individual task durations, but the application should calculate the final range using code.

Example:

```
type TimeEstimate = {
  minimumMinutes: number;
  maximumMinutes: number;
  explanation: string[];
};

export function estimateCleaningTime(
  taskMinutes: number[],
  roomSize: "small" | "medium" | "large",
  intensity: "quick" | "standard" | "deep"
): TimeEstimate {
  const base = taskMinutes.reduce((sum, value) => sum + value, 0);

  const sizeMultiplier = {
    small: 0.9,
```

```

    medium: 1,
    large: 1.25,
  }[roomSize];

  const intensityMultiplier = {
    quick: 0.7,
    standard: 1,
    deep: 1.5,
  }[intensity];

  const adjusted = base * sizeMultiplier * intensityMultiplier;

  return {
    minimumMinutes: Math.max(5, Math.round(adjusted * 0.8)),
    maximumMinutes: Math.max(10, Math.round(adjusted * 1.25)),
    explanation: [
      `Adjusted for a ${roomSize} room.`,
      `Adjusted for ${intensity} cleaning.`,
      "Actual time depends on pace and hidden clutter.",
    ],
  };
}

```

The exact multipliers will need tuning. They should not be presented as scientifically validated measurements.

Features

- calculate total task time
- account for room size
- account for cleaning intensity
- add uncertainty buffer
- round output to sensible five-minute intervals
- cap unreasonable estimates
- show assumptions

Technology used

- TypeScript
- pure functions
- Vitest parameterized tests

Why this matters for his resume

This demonstrates that he did not simply send an image to an API and print the response. He created domain logic, validation and deterministic processing around an unreliable model.

Step 5: Add task ordering

Tasks should appear in a practical sequence rather than whatever order the model returns.

A reasonable general order is:

1. safety hazards
2. trash
3. dishes
4. laundry
5. items that belong elsewhere
6. organization
7. surface cleaning
8. floor cleaning

Floor cleaning should usually be last because earlier tasks may drop dust or debris.

Implement a priority map:

```
const categoryOrder = {  
  trash: 10,  
  dishes: 20,  
  laundry: 30,  
  organizing: 40,  
  surface_cleaning: 50,  
  floor_cleaning: 60,  
  other: 70,  
};
```

Tasks can then be sorted using:

- safety priority
- category order
- dependency
- estimated effort

Technology used

- TypeScript
- deterministic sorting
- unit tests

Optional enhancement

Allow the AI to suggest dependencies, but validate them in code:

```
{  
  "taskId": "vacuum-floor",  
  "dependsOn": ["pick-up-clothes", "remove-trash"]  
}
```

Do not build a complicated graph engine for the MVP.

Phase 4: Build the web interface

Step 6: Create the Next.js project

Initialize the project with:

- TypeScript
- App Router
- ESLint
- Tailwind CSS
- `src` directory, if preferred
- import aliases

Next.js provides the application routing, server-side route handlers and React UI within one project. ([Next.js](#))

Initial pages

/	Landing and upload page
/analyze	Analysis progress page or state
/results	Results view
/about	Project explanation and limitations

For the first implementation, `/analyze` and `/results` can be states on the home page. Separate routes are useful later for shareable analyses.

Technology used

- Next.js App Router
 - React
 - TypeScript
 - Tailwind CSS
 - shadcn/ui
 - ESLint
 - Prettier
-

Step 7: Build the image-upload component

The uploader should support:

- file selection
- drag and drop

- phone camera capture
- image preview
- replacing the image
- removing the image
- visible validation errors

On mobile, use:

```
<input
  type="file"
  accept="image/jpeg,image/png,image/webp"
  capture="environment"
/>
```

Validation rules

Validate both client-side and server-side:

- allowed MIME types
- maximum file size
- minimum usable dimensions
- exactly one image
- no SVG
- no arbitrary file extensions
- gracefully handle corrupted images

Do not trust only the filename extension.

Privacy notice

Before upload, show:

Avoid uploading images containing people, addresses, financial documents, medical information or other sensitive personal details.

Also tell the user whether the image will be stored or deleted.

Technology used

- React client component
- native browser File API
- TypeScript
- Zod
- optional image resizing with a browser image-compression library

MVP storage decision

For the first version, avoid permanent storage. Send the image for analysis and discard it after processing.

That is simpler and more privacy-conscious than immediately building an image library.

Step 8: Build the preferences form

After selecting an image, ask only a few questions:

- room type
- room size
- cleaning intensity
- preferred music genres
- explicit-content preference
- available cleaning time, optionally

Example:

Room: Bedroom

Size: Medium

Goal: Standard cleaning

Music: Pop, Tamil, Electronic

Available time: 40 minutes

Why these inputs matter

They reduce the amount the AI needs to guess and make the recommendation more useful.

Technology used

- React Hook Form
 - Zod
 - shadcn/ui form components
-

Phase 5: Connect the frontend to the AI

Step 9: Create the analysis API route

Create:

`app/api/analyze/route.ts`

Responsibilities:

1. Accept multipart form data.
2. Validate the uploaded image.
3. Validate user preferences.

4. Reject oversized or unsupported requests.
5. Call the room-analysis service.
6. Validate the model's output.
7. calculate final time range
8. order tasks
9. return a stable response contract
10. hide all API keys from the browser

Next.js Route Handlers are server-side HTTP handlers defined inside the App Router, so the OpenAI key can remain on the server. ([Next.js](#))

API response

```
{
  "analysisId": "temporary-id",
  "room": {
    "type": "bedroom",
    "clutterLevel": 7,
    "summary": "Clothing and loose objects cover part of the floor..."
  },
  "timeEstimate": {
    "minimumMinutes": 30,
    "maximumMinutes": 45
  },
  "tasks": [],
  "musicProfile": {
    "energy": "high",
    "mood": "motivating",
    "genres": ["pop", "dance"],
    "targetBpmMin": 120,
    "targetBpmMax": 135
  },
  "assumptions": [],
  "confidence": 0.76
}
```

Error responses

Use predictable codes:

```
{
  "error": {
    "code": "UNSUPPORTED_IMAGE",
    "message": "Please upload a JPEG, PNG or WebP room photo."
  }
}
```

Potential error codes:

- INVALID_REQUEST
- FILE_TOO_LARGE
- UNSUPPORTED_IMAGE
- IMAGE_UNREADABLE
- NOT_A_ROOM
- AI_ANALYSIS_FAILED
- INVALID_AI_RESPONSE
- RATE_LIMITED
- INTERNAL_ERROR

Technology used

- Next.js Route Handler
 - OpenAI SDK
 - Zod
 - TypeScript
 - server-only environment variables
-

Step 10: Build analysis loading and failure states

AI requests can take several seconds. The interface must not appear frozen.

Show stages such as:

Uploading image...

Inspecting visible clutter...

Creating cleaning tasks...

Estimating the schedule...

Finding music...

These stages may be approximate UI progress; do not falsely imply that each exact backend operation has completed unless it has.

Add:

- cancel or reset option
- timeout handling
- retry button
- friendly error details
- no duplicate submissions
- disabled button during request

Technology used

- React state
- AbortController

- Next.js client components
 - accessible progress indicator
-

Phase 6: Build the results experience

Step 11: Display the room analysis

The results screen should show:

Overview

- image thumbnail
- room type
- clutter level
- estimated time
- confidence
- cleaning intensity

Observations

- clothes visible on the floor
- desk covered with loose items
- trash near the bed
- floor partially obstructed

Assumptions and limitations

- only visible areas were analyzed
- hidden clutter was not considered
- actual time varies by cleaning speed

Avoid

Do not display a humiliating score such as:

Your room is 91% disgusting.

Use neutral language:

- lightly cluttered
- moderately cluttered
- heavily cluttered
- limited visibility
- potentially time-consuming

Technology used

- React

- Tailwind CSS
 - shadcn/ui cards and progress components
-

Step 12: Build the interactive cleaning checklist

Each task should include:

- checkbox
- task title
- short reason
- estimated time
- category
- priority
- edit button
- delete button

Example:

- ☐ Collect clothing from floor 5–8 min
Clears the walking area before vacuuming.
- ☐ Remove cups and food containers 3–5 min
Prevents spills and frees the desk surface.
- ☐ Organize desk items 10–15 min
Group electronics, papers and miscellaneous items.
- ☐ Vacuum exposed floor 5–10 min
Complete this after objects have been removed.

Add:

- progress percentage
- completed minutes
- remaining estimated time
- “Start cleaning” mode
- reset checklist
- local persistence in the browser

Technology used

- React state
- `localStorage`
- TypeScript
- optional `useReducer` for task state

Why local storage first

A database and user account system are unnecessary merely to preserve one checklist on the user's device.

Step 13: Add focus mode

When the user presses **Start Cleaning**, show one task at a time.

Example:

Task 2 of 7

Remove dishes and cups

Estimated time: 4 minutes

[Start timer] [Mark complete] [Skip]

Features:

- countdown or count-up timer
- next task
- pause
- skip
- completed progress
- updated remaining-time range

Technology used

- React state
- browser timer APIs
- localStorage

This makes the app a real cleaning assistant rather than a static AI report.

Phase 7: Add music recommendations

Step 14: Convert cleaning analysis into a music profile

Do not ask the AI to name random songs. First derive a structured music profile:

```
{  
  "sessionLengthMinutes": 42,  
  "energy": "high",  
  "mood": "motivating",  
  "genres": ["pop", "dance"],  
  "targetBpmMin": 120,
```

```
"targetBpmMax": 135,  
"explicitAllowed": false  
}
```

Logic could be:

- under 15 minutes → short, energetic reset
- 15–40 minutes → steady, upbeat
- over 40 minutes → varied playlist with warm-up and sustained energy
- detailed organizing → moderate-energy, less distracting
- physical cleaning → higher-energy music

Technology used

- TypeScript rules
- AI result as input
- unit tests

The mapping should primarily be deterministic. The AI may suggest mood, but the application should own the final rule set.

Step 15: Integrate Spotify search

For the MVP, search for existing public playlists rather than creating a new playlist.

Search terms might be built from:

high energy pop cleaning
upbeat dance workout clean
40 minute focus cleaning
Tamil energetic hits

Spotify's search endpoint supports searching playlist catalog data, and Spotify API requests require authorization. ([Spotify for Developers](#))

Backend endpoint

app/api/music/route.ts

Responsibilities:

1. obtain a Spotify application token
2. construct a search query
3. search playlists
4. filter invalid or incomplete results
5. return three to five playlist options

6. cache results temporarily
7. return Spotify links and attribution

Playlist card

Display:

- cover art
- playlist name
- owner
- Spotify link
- approximate relevance explanation

Spotify requires linked attribution when displaying Spotify metadata and imagery, so playlist cards should link back to Spotify. ([Spotify for Developers](#))

Important limitation

Do not send Spotify's music catalog or content into an AI model for training or analysis. Spotify explicitly restricts using Spotify content to train or otherwise ingest into machine-learning or AI models. The safe architecture is:

AI produces generic music criteria

|

v

Application builds search terms

|

v

Spotify search returns playlists

Do not route Spotify data back through the AI. ([Spotify for Developers](#))

Technology used

- Spotify Web API
- OAuth Client Credentials flow for catalog search
- server-side `fetch`
- TypeScript
- Next.js Route Handler

Phase 8: Improve reliability and security

Step 16: Add security controls

At minimum:

- keep API keys server-side
- limit image size
- restrict MIME types
- sanitize filenames
- rate-limit analysis requests
- impose request timeouts
- validate every AI response
- avoid logging raw images
- avoid exposing full AI errors to users
- do not store sensitive uploads by default
- delete temporary images
- configure environment variables correctly

Rate limiting

For a portfolio app, even a simple per-IP limit is useful:

5 analyses per hour per IP

Without this, a public visitor or bot could consume the entire AI budget.

Technology used

- Vercel environment variables
- Upstash Redis rate limiting, later
- Zod
- server-only modules
- structured logging

For the first private demo, an in-memory limiter may be acceptable. For a publicly shared deployment, use durable rate limiting.

Step 17: Add image moderation and content handling

The app needs to handle:

- images containing people
- inappropriate content
- non-room images
- documents containing private information
- extremely dark or blurry images

Possible behavior:

We couldn't confidently analyze this as a room photo.
Please upload a clear image showing most of the room.

OpenAI provides image-capable moderation, which can classify potentially harmful image input. ([OpenAI Platform](#))

Technology used

- image moderation endpoint or model-supported safety handling
 - custom room-image validity check
 - server-side validation
-

Step 18: Add structured logging

Log metadata, not sensitive image content.

Good log record:

```
{
  "event": "room_analysis_completed",
  "requestId": "req_123",
  "durationMs": 4821,
  "imageSizeBytes": 1438290,
  "taskCount": 7,
  "confidence": 0.78,
  "model": "configured-model-name",
  "success": true
}
```

Do not log:

- base64 image contents
- raw uploaded photographs
- API tokens
- full authorization headers
- personal information inferred from images

Technology used

- console-based structured JSON logs initially
 - Vercel Logs
 - optional Sentry later
-

Phase 9: Testing

Step 19: Unit tests

Test:

- schema validation
- time calculations
- task ordering
- music-profile mapping
- Spotify search-query generation
- file validation
- error mapping

Technology used

- Vitest
-

Step 20: Component tests

Test:

- image preview
- invalid file message
- preferences form
- checklist interaction
- editing and deleting tasks
- progress updates
- playlist rendering

Technology used

- React Testing Library
 - Vitest
 - user-event
-

Step 21: API integration tests

Mock external APIs and test:

- valid AI response
- invalid AI response
- OpenAI timeout
- OpenAI rate limit
- Spotify failure
- malformed image
- oversized upload
- non-room image

Do not call paid live APIs in every CI build.

Technology used

- Vitest
 - Mock Service Worker or dependency injection
 - fixture JSON responses
-

Step 22: End-to-end tests

Test the complete browser flow:

1. open the homepage
2. upload a sample room
3. choose preferences
4. submit
5. wait for mocked analysis
6. inspect results
7. complete a task
8. open Spotify recommendation
9. reload and verify local state

Technology used

- Playwright
-

Step 23: Create an evaluation harness for the AI

Ordinary unit tests are not enough because AI responses are probabilistic.

Maintain a small test dataset:

```
evals/  
├── images/  
│   ├── bedroom-light-clutter.jpg  
│   ├── bedroom-heavy-clutter.jpg  
│   ├── kitchen-dishes.jpg  
│   ├── office-papers.jpg  
│   └── invalid-landscape.jpg  
├── expected/  
│   └── evaluation-criteria.json  
└── run-evals.ts
```

Do not require exact output equality. Evaluate:

- valid schema
- correct room category
- no invented dangerous tasks
- reasonable number of tasks

- evidence grounded in visible image
- time range not absurd
- uncertainty included
- non-room image rejected

Technology used

- TypeScript
- custom evaluation script
- stored test fixtures
- optional model grader later

This is one of the most valuable engineering elements on the resume because it shows awareness that AI features require evaluation, not just prompts.

Phase 10: Deployment

Step 24: Deploy to Vercel

Deployment tasks:

1. push repository to GitHub
2. import project into Vercel
3. configure OpenAI environment variable
4. configure Spotify client credentials
5. deploy preview environment
6. verify API routes
7. verify mobile image upload
8. set production domain
9. set spending and usage alerts
10. test production rate limiting

Vercel is a natural deployment target for a Next.js application and supports server-side execution and environment configuration. ([Next.js](#))

Technology used

- GitHub
 - Vercel
 - Vercel environment variables
 - GitHub Actions optionally
-

Step 25: Add CI

On every pull request, run:

```
npm run lint  
npm run typecheck  
npm run test  
npm run build
```

Run Playwright tests either:

- against mocked services during pull requests, or
- against a Vercel preview deployment

Technology used

- GitHub Actions
 - ESLint
 - TypeScript compiler
 - Vitest
 - Playwright
-

Phase 11: Portfolio polish

Step 26: Write a strong README

The README should contain:

- product overview
- live demo
- screenshots
- architecture diagram
- feature list
- technical stack
- setup instructions
- environment variables
- testing strategy
- AI evaluation strategy
- privacy decisions
- known limitations
- future roadmap

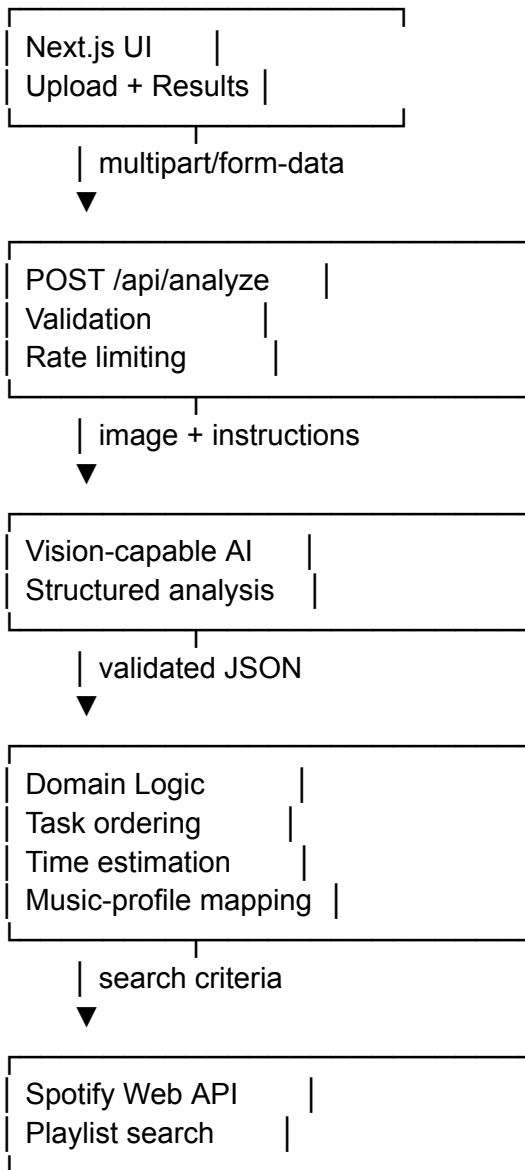
It should explicitly explain:

AI output is schema-validated, task timing is adjusted through deterministic application logic, and room photographs are not permanently stored by default.

That sentence separates the project from a shallow “API wrapper.”

Step 27: Add an architecture diagram

Example:



Features to add only after the MVP

Enhancement 1: Before-and-after comparison

The user uploads a second image after cleaning.

The app compares:

- visible floor space
- reduction in loose objects
- surface visibility
- completed task categories
- estimated improvement

It should not rely on exact pixel-perfect image alignment.

Technology

- same vision model
 - comparison schema
 - before/after result page
 - optional short-term image storage
-

Enhancement 2: Saved cleaning sessions

Add accounts only when saved history becomes necessary.

Data model:

User
CleaningSession
RoomAnalysis
CleaningTask
MusicRecommendation
BeforeAfterComparison

Recommended technology

- Supabase Auth
- PostgreSQL through Supabase
- Prisma or Drizzle ORM

Do not use MongoDB merely because the app involves AI. The data is relational enough for PostgreSQL.

Enhancement 3: Private image storage

If saved room images are introduced, use private object storage rather than public URLs.

Vercel Blob supports runtime file uploads and private storage, but uploaded images should be deleted when no longer needed. ([Vercel](#))

Recommended technology

- Vercel Blob private storage, or
- Supabase Storage private bucket

Do not add image persistence in the first release unless the before-and-after feature requires it.

Enhancement 4: Spotify account connection

Later, users could sign in to Spotify and create their own cleaning playlist.

That requires:

- Spotify Authorization Code flow
- user consent
- playlist modification scopes
- token refresh handling
- playlist creation
- adding selected tracks

Spotify supports creating playlists and adding items, but those actions require user authorization and playlist modification scopes. ([Spotify for Developers](#))

This is a legitimate second-stage feature, not an MVP requirement.

Enhancement 5: Cleaning-session adaptation

Let users select:

- “I have only 15 minutes”
- “Give me the highest-impact tasks”
- “Skip organizing; focus on sanitation”
- “I have limited mobility”
- “No vacuuming right now”

The backend can select tasks using a small optimization function:

maximize task impact

subject to total duration $\leq$ available minutes

That would make a strong algorithmic enhancement.

Exact recommended build order

Order	Feature	Primary technology
1	Product requirements and response contract	Markdown, TypeScript
2	Zod analysis schema	Zod, TypeScript
3	Local AI image-analysis experiment	OpenAI SDK, Node.js
4	AI evaluation image set	TypeScript, fixture images
5	Deterministic time estimator	TypeScript, Vitest
6	Task-ordering logic	TypeScript, Vitest
7	Next.js application shell	Next.js, React, Tailwind
8	Image upload and preview	React, File API
9	Preferences form	React Hook Form, Zod
10	<code>/api/analyze</code> backend	Next.js Route Handler
11	Analysis progress and errors	React
12	Results dashboard	React, Tailwind
13	Editable cleaning checklist	React, localStorage
14	Focus mode and timer	React
15	Music-profile rules	TypeScript
16	Spotify playlist search	Spotify Web API
17	Privacy and moderation controls	Server validation
18	Rate limiting and logging	Upstash/Vercel
19	Unit and component tests	Vitest, RTL
20	End-to-end tests	Playwright
21	Deployment	GitHub, Vercel
22	README and architecture documentation	Markdown, Mermaid
23	Before-and-after comparison	Vision AI
24	Accounts and saved sessions	Supabase, PostgreSQL
25	Spotify account integration	Spotify OAuth

Recommended stopping point for the first release

He should stop the first release after Step 22.

That version will already demonstrate:

- multimodal AI
- structured model outputs
- API integration
- backend route design
- input validation
- deterministic business logic
- third-party music integration
- privacy handling
- automated testing
- deployment
- technical documentation

That is enough for a serious portfolio project. Adding authentication, databases and social features before completing this core would be scope creep.

The project's strongest engineering story should be:

"I built a multimodal planning system that validates AI-generated room analysis against a strict schema, converts it into a deterministic cleaning workflow, estimates a bounded schedule, maps the workload to a music profile, and integrates with Spotify—all while avoiding permanent storage of user images by default."